

駕馭急性子 AI 助手

The Michelin-Star AI
Kitchen Blueprint

```
<html>
<head>
  <title>AI Kitchen</title>
  <meta charset="utf-8">
  <script src="/js/main.js"></script>
</head>
<body>
  <div class="container">
    <div class="header">
      <h1>AI Kitchen</h1>
    </div>
    <div class="main">
      <div class="chef">
        <img alt="AI Chef" data-bbox="250 450 350 550"/>
      </div>
      <div class="ingredients">
        <ul>
          <li>Ingredients</li>
          <li>Ingredients</li>
          <li>Ingredients</li>
        </ul>
      </div>
      <div class="recipe">
        <h2>Recipe</h2>
        <ol>
          <li>Step 1</li>
          <li>Step 2</li>
          <li>Step 3</li>
        </ol>
      </div>
    </div>
  </div>
</body>
</html>
```

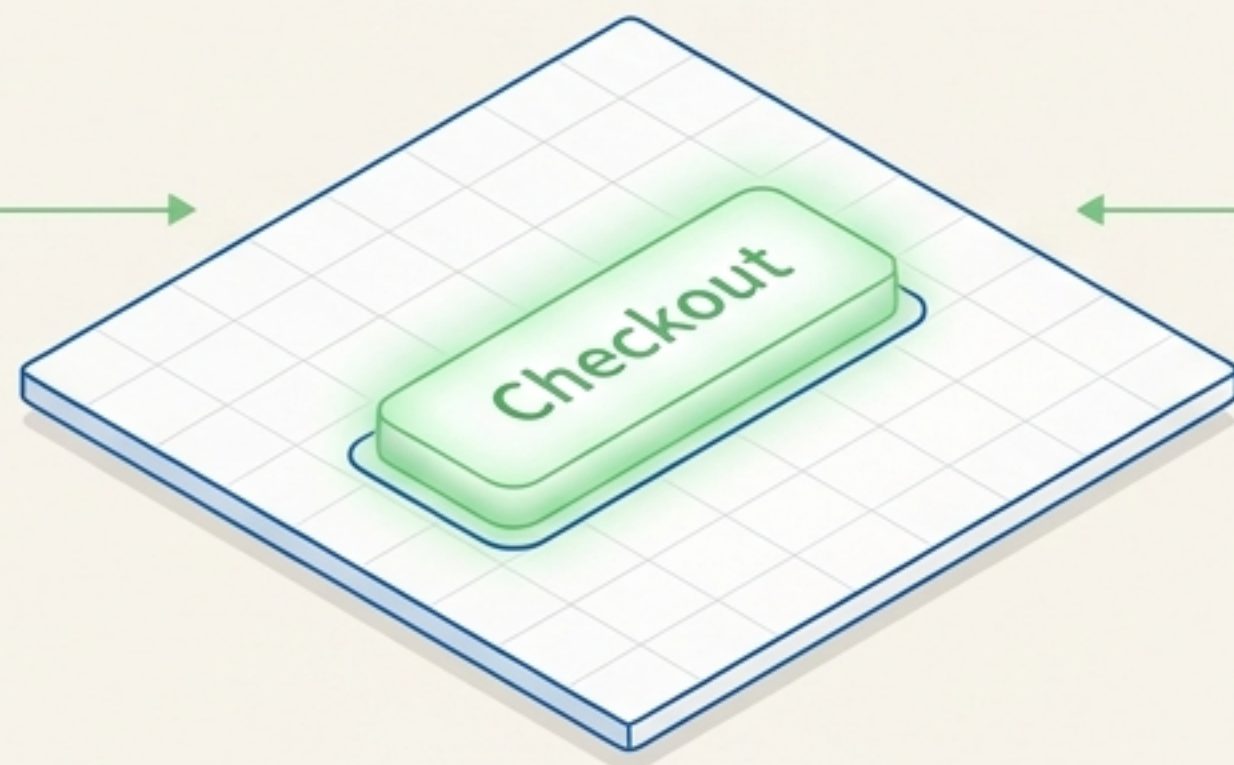
```
1 object {
2   name: "AI Kitchen",
3   description: "A Michelin-Star AI Kitchen",
4   author: "AI Kitchen",
5   version: "1.0.0",
6 }
7 }
```

```
1 1.0.0
2 1.0.0
3 1.0.0
4 1.0.0
5 1.0.0
6 1.0.0
7 1.0.0
8 1.0.0
9 1.0.0
```

```
$ ce-sdk
$ cd -b node
>
```

AI 最常見的失敗模式不是笨，而是「太急著表現」

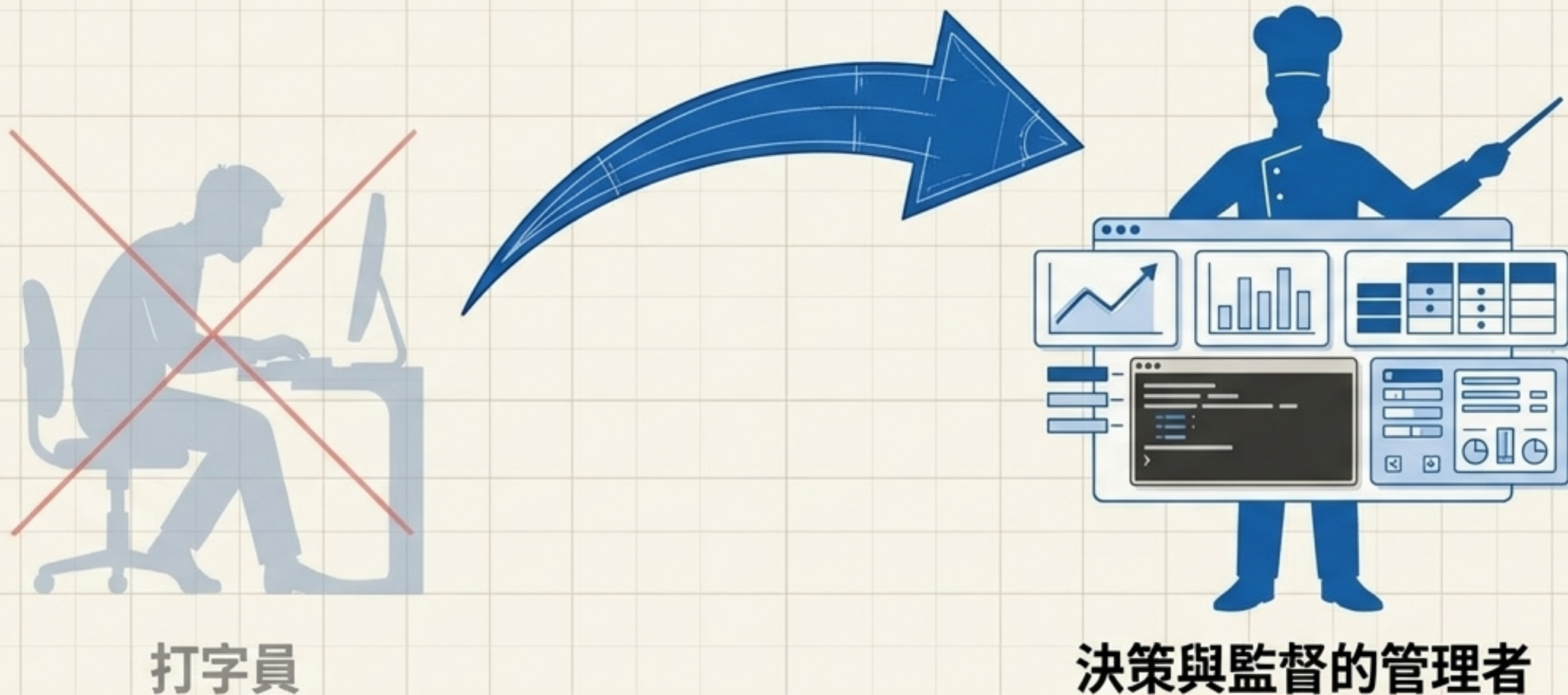
幫我修一下購物車的結帳按鈕。



改了 400 行程式碼，
連線設定全部被清空

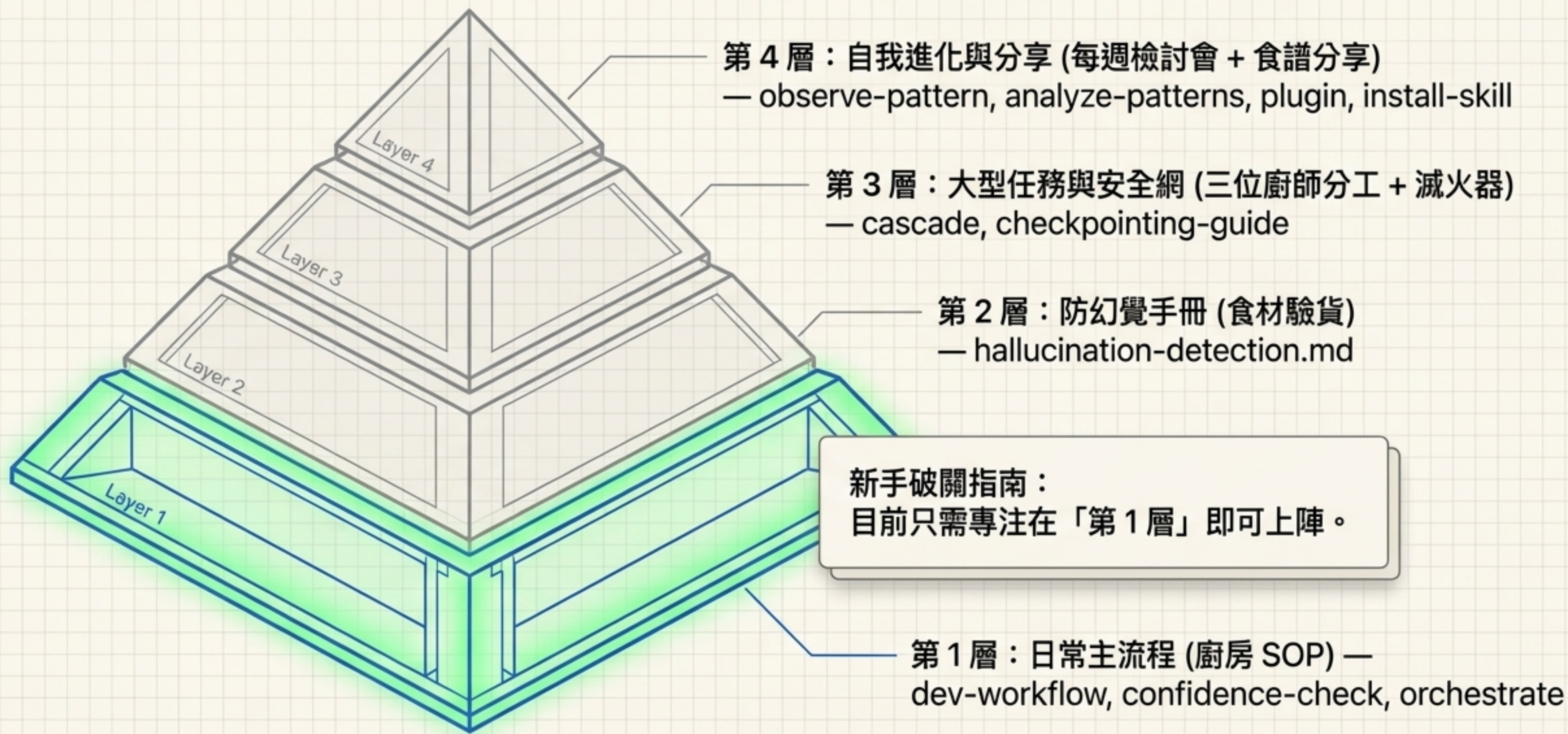
致命痛點：需求還沒搞清楚，就開始狂改檔案。
這就是「過早開始寫程式」的災難。

寫程式的時代變了，你現在是這支 AI 團隊的決策管理者

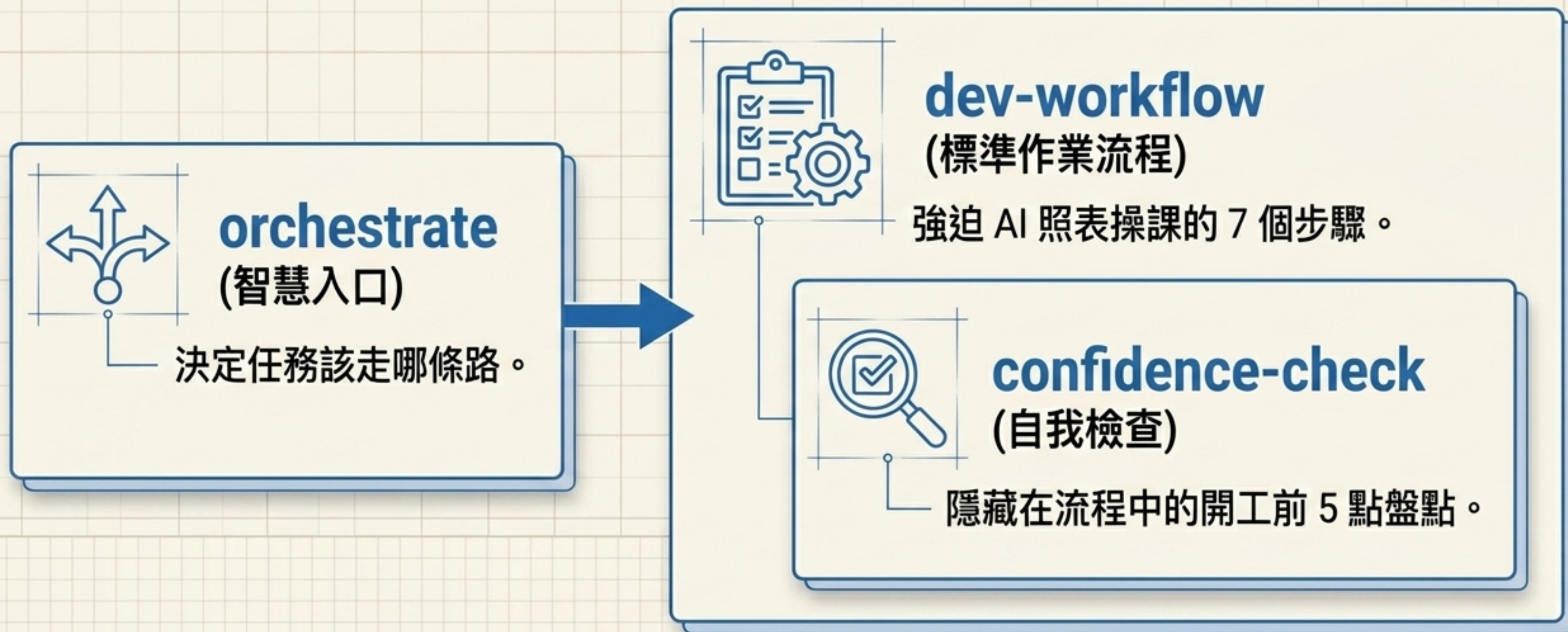


第七章的 8 個工具不是用來「幫你寫程式」的，
而是一套「管住這位急性子助手」的管理工具箱。

AI 現代無塵廚房的四層系統藍圖



每天都會用到的第一層：日常主流程

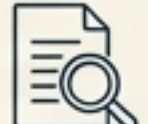


dev-workflow：強迫 AI 照表操課的七階段流程



> /dev-workflow

confidence-check：動手寫程式前的五項自我檢查清單

☐	C1: 重複實作檢查 	防： 重寫公司已有的模組
☒	C2: 架構合規檢查 	防： 違反團隊分層規定
☒	C3: 官方文件檢查 	防： 使用不存在的舊版 API
☒	C4: 開源參考檢查 	防： 重新發明輪子
☒	C5: 根本原因確認 	防： 沒搞懂 Bug 就瞎改

出門前的口訣「鑰匙、錢包、手機」。少帶一樣可能沒事，少帶兩樣今天一定出事。

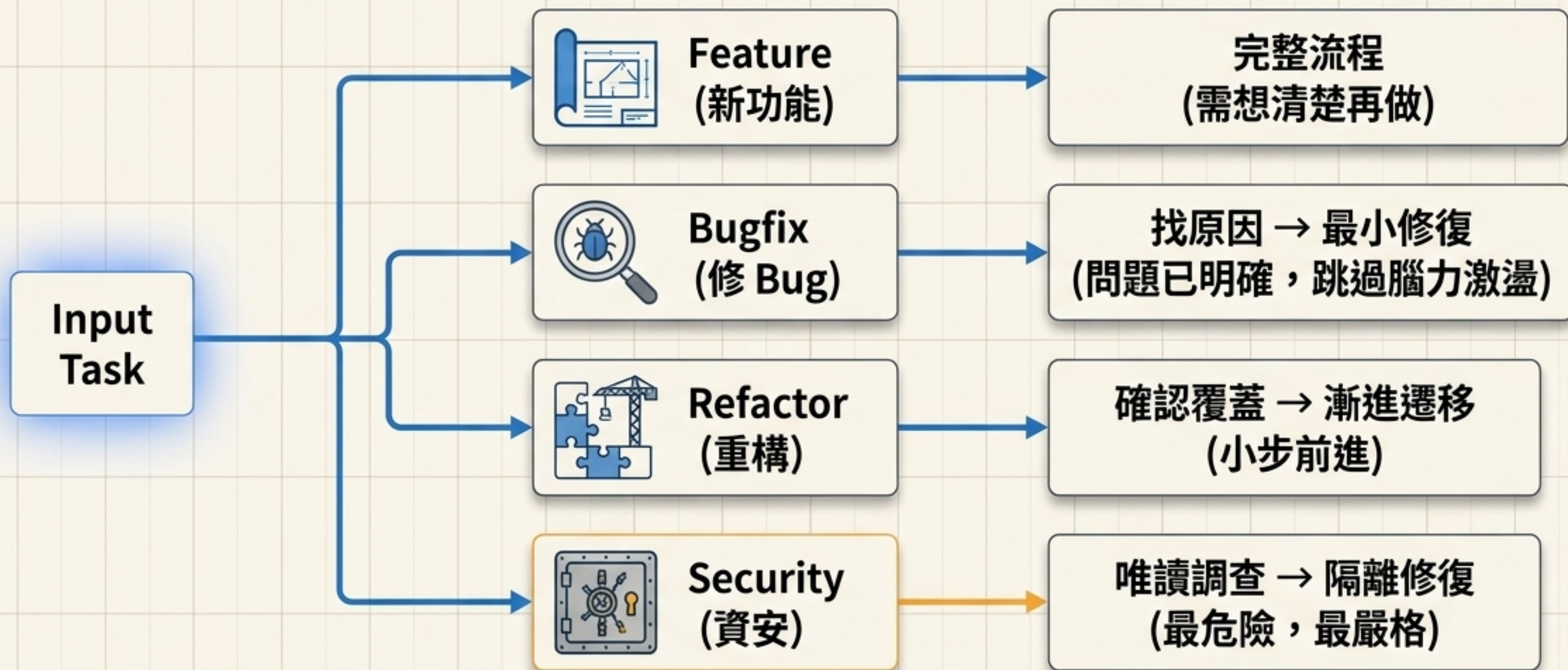
90%
Pass Rate
Required

Rule: 5 題至少答對 4 題才准開始寫程式。

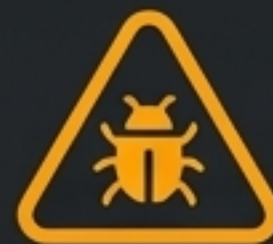
orchestrate：根據任務規模自動分流的智慧檢傷站



醫院的檢傷分類。感冒不用進開刀房，
但心臟病發不能只給你普拿疼。



修復 Bug 的鐵律：寫不出失敗測試，就鎖死不准碰程式碼



鐵律：AI 在寫任何修復程式碼之前，必須先寫出一個「能重現這個 Bug 的失敗測試」。

Why: 寫不出失敗測試，代表你根本不知道問題在哪，後面全是瞎猜。

hallucination-detection：逼退 AI 幻覺的四個靈魂拷問

當 AI 瞎掰不存在的東西時使用。這是一份給人類看的手冊。

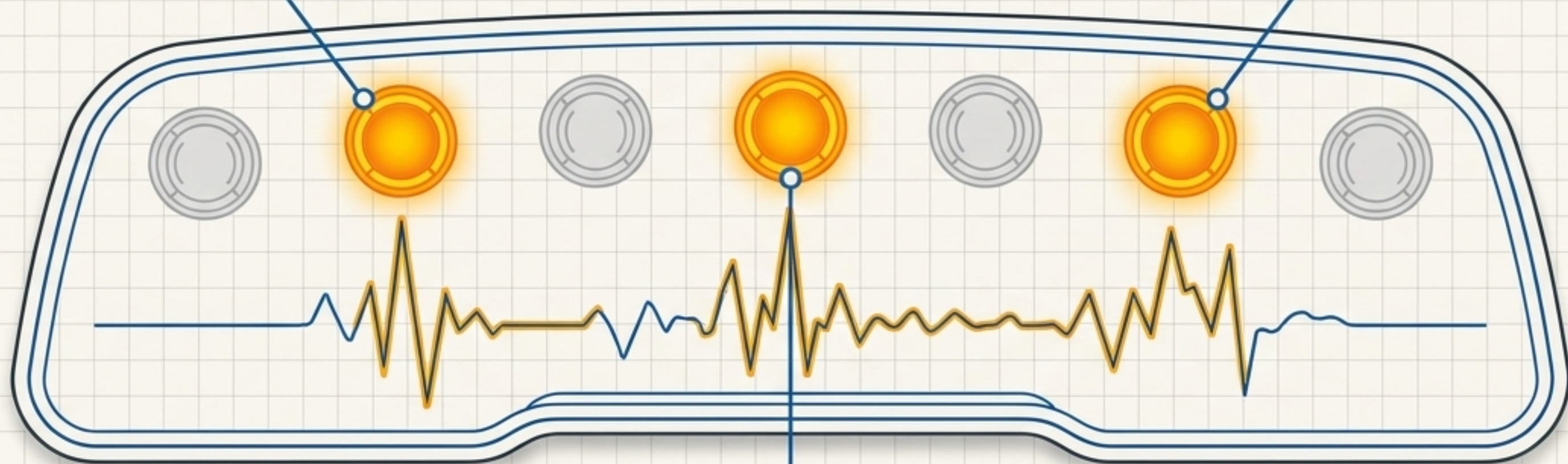


核心邏輯：捏造程式碼很容易，但捏造「點得開的網址」幾乎不可能。逼它拿出實體證據。

聽懂 AI 的心虛：識破幻覺的七個紅旗警示燈

「應該是...」「大概...」「似乎...」
將猜測當事實講

「我相信這是正確的」伴隨大量複雜術語
典型的自信滿滿式心虛



「效能提升快 30%」
沒有出處的數字



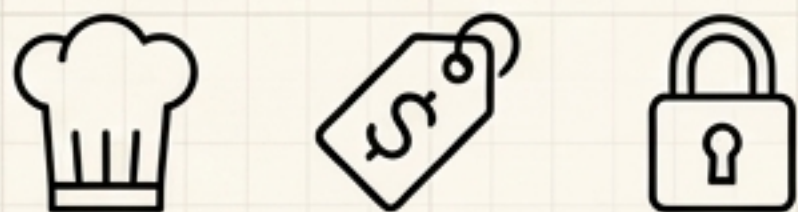
Actionable Advice: 看到這些訊號，不要爭論，直接砸出「四個靈魂拷問」。

cascade-start：讓三位 AI 廚師在隔離環境中各司其職

Problem: 超大型任務只靠一個 AI，做到一半一定會「斷片」。

Capability/Role Matrix

Worker 1 (主廚/架構師)



最高智商 (最貴)
權限：只能看、只能指揮，
不能寫程式。

Worker 2 (二廚/實作者)



速度與能力均衡
權限：完整讀寫，負責真
正寫程式。

Worker 3 (品管/驗證員)



最快最便宜
權限：只能讀檔案和跑測
試。

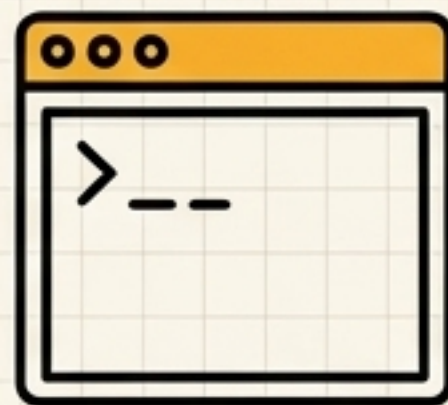
三人永遠不會互相弄髒對方的工作檯。

checkpointing-guide：時光機的安全區與終端機破壞的致命陷阱

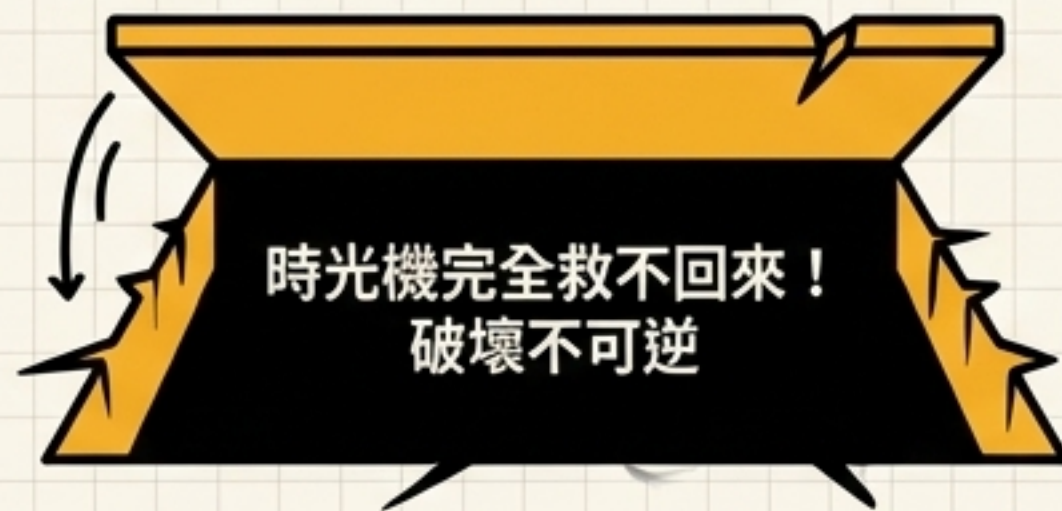


AI 用檔案編輯工具改的文字檔

連按兩次 Esc 可無損還原



終端機指令（如 `npm install` 或搬移資料夾）



Solution：必須採取雙重保險策略（Git 備份 + Checkpoint 雙軌並行）。

observe & analyze：行車記錄器與每週檢討會的自動優化迴圈



紀錄動作 (observe-pattern.sh)

AI 每做一個動作，默默記一筆到日誌檔 patterns.jsonl。

Impact：把「只有老鳥才知道的眉角」
變成白紙黑字的自動制度。



找出規律 (analyze-patterns)

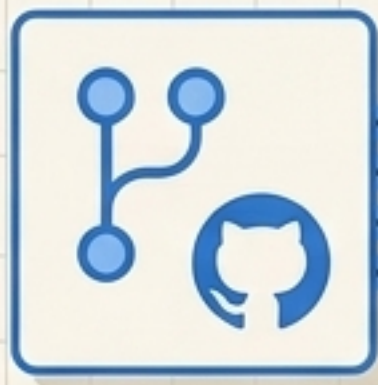
定期分析日誌。例如：發現同一指令被拒絕 5 次，主動建議修改設定。

plugin & install：將個人秘方打包分享至團隊生態系

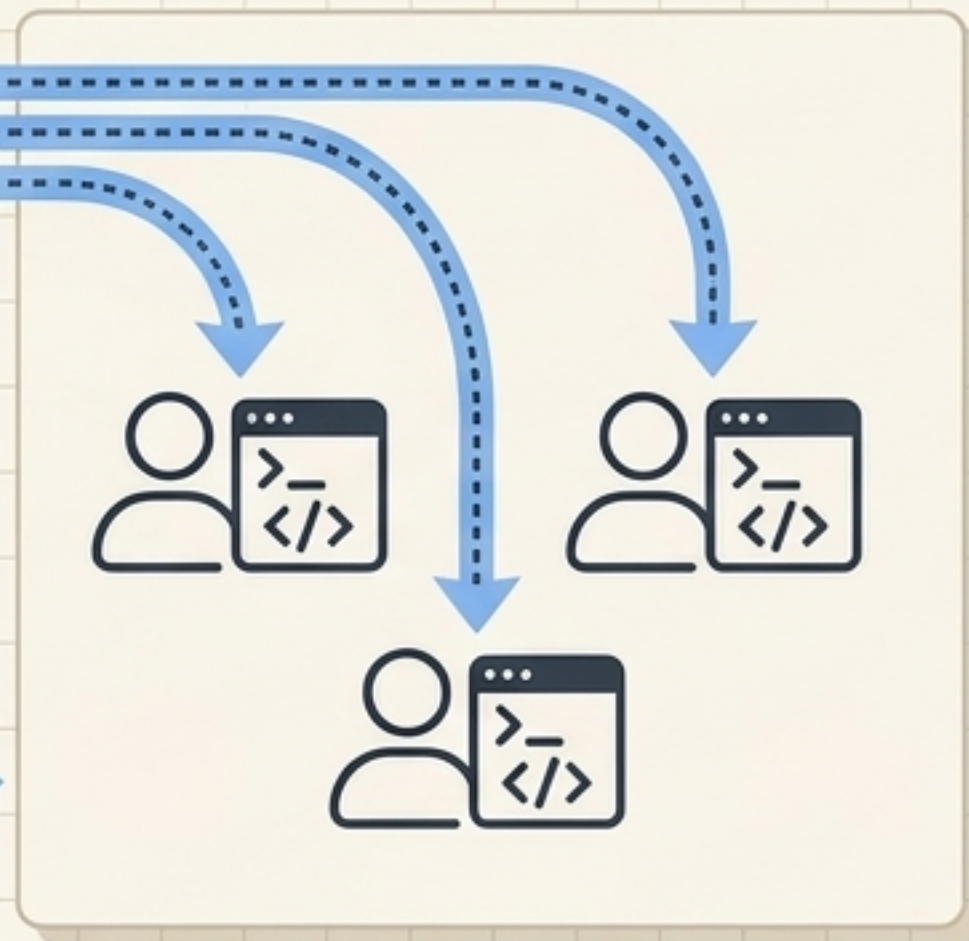
打包出去 (plugin-example)



將調教好的 Skills + Hooks
打包成外掛。



安裝進來 (install-skill.sh)



同事只需一行指令，立刻擁有相同的
米其林廚房配置。

Impact: 經驗不再侷限於個人桌面，而是整個社群的資產。

新手上路指南：三週漸進式解鎖計畫

不要一次全學！循序漸進才是最快的方法。

第一週 (建立紀律)

導入第 1 層。專注熟練 /dev-workflow 與 /orchestrate，習慣動手前的檢查。

第二週 (防禦與規模化)

閱讀 hallucination-detection.md，學習辨識 AI 幻覺；嘗試用 cascade-start 處理稍大任務。

第三週+ (系統進化)

啟動 observe-pattern 收集習慣，開始打包專屬的團隊 plugin。

常見疑問：為什麼有兩份一模一樣的設定檔？

Question: 為什麼 confidence-check 和 orchestrate 在資料夾裡各有兩份？

skills/ (實際運作的引擎)

```
├── .claude/
│   └── skills/
│       ├── confidence-check/
│       ├── orchestrate/
│       └── ...
```

安裝後該有的樣子 (直接複製到專案就能用)。

根目錄 (展示用的櫥窗)

```
├── /
│   ├── confidence-check/
│   ├── orchestrate/
│   └── ...
```

書中展示用的副本。

Resolution: 兩份內容完全相同，直接複製 skills/ 內的檔案即可，無需糾結。

從暴走的產生器，蛻變為一支有紀律的工程團隊

Core Thesis: 這 8 樣工具的本質，不是為了限制 AI 的才華，而是賦予它紀律。



Final Takeaway: 掌握這套藍圖，你就不再只是打字的人，而是能真正駕馭 AI 的高階決策管理者。